



Solution Design Document

ACDC - Card Fields

Installments (MSI / MCI) · 3DS Risk Initiated · Vault · Fraudnet · STC

Tipo de documento	Solution Design Document (SDD)
Solución	Procesamiento de tarjetas de crédito y débito en checkout web mediante PayPal ACDC
Componentes técnicos	PayPal JavaScript SDK · Card Fields · Vault v3 · Calculated Financing Options · STC · Fraudnet
Capacidades funcionales	Cobro con tarjeta nueva · Vault-with-Purchase · Cobro con token · Installments (MSI/MCI) · 3DS Risk Initiated · Fraudnet + STC
APIs REST involucradas	/v1/oauth2/token · /v2/checkout/orders · /v3/vault/payment-tokens · /v1/credit/calculated-financing-options · /v1/risk/transaction-contexts
Convenciones	REST → snake_case · SDK JavaScript → camelCase
Audiencia	Solution Architecture · Integration Engineering · Product · e-commerce · Riesgo · Cumplimiento PCI

Tabla de Contenidos

Tabla de Contenidos	2
1. Resumen Ejecutivo.....	6
2. Contexto de Negocio y Drivers.....	7
2.1 Drivers de negocio	7
2.2 Drivers regulatorios.....	7
3. Alcance de la Solución	8
3.1 Capacidades en alcance.....	8
3.2 Capacidades fuera de alcance.....	8
4. Stakeholders y Audiencia.....	10
5. Glosario de Términos y Acrónimos.....	11
6. Visión General de la Solución.....	13
6.1 Componentes de la solución.....	13
6.2 Identificador de correlación: el CMID	14
7. Requisitos Funcionales.....	15
8. Requisitos No Funcionales	16
9. Arquitectura de la Solución	17
9.1 Diagrama lógico	17
9.2 Reglas no negociables de la arquitectura	17
9.3 Frontera de seguridad por asset.....	18
10. Prerrequisitos y Configuración del Entorno	19
10.1 Habilitaciones comerciales requeridas	19
10.2 Credenciales y configuración del backend	19
10.3 Requisitos del entorno.....	19
11. Autenticación OAuth2 y Emisión del id_token.....	21
11.1 Request.....	21
11.2 Response.....	21
11.3 Caché del access_token.....	21
11.4 Endpoint expuesto al frontend	22
11.5 Implementación de referencia (Node.js)	22
12. Generación del Client Metadata ID (CMID).....	23
12.1 Reglas de ciclo de vida	23
12.2 Implementación de referencia.....	23
12.3 Validación.....	23

13. Integración de Fraudnet.....	25
13.1 Script de configuración (JSON).....	25
13.2 Script de la librería.....	25
13.3 Implementación dinámica	25
13.4 Reglas operativas	26
14. Set Transaction Context (STC).....	27
14.1 Endpoint	27
14.2 Body — set genérico para Retail	27
14.3 Referencia de campos.....	27
14.4 Manejo de respuesta — comportamiento no bloqueante.....	28
14.5 Implementación de referencia	29
15. Carga del JavaScript SDK de PayPal	30
15.1 Construcción de la URL del SDK	30
16. Estructura HTML y Estilos de Card Fields	31
16.1 Contenedores del DOM	31
16.2 Renderizado	31
16.3 Estilos — dos capas independientes.....	31
17. Creación y Captura de Órdenes.....	33
17.1 Headers HTTP obligatorios	33
17.2 Create Order — payload completo (tarjeta nueva con Vault)	33
17.3 Reglas de validación del breakdown.....	34
17.4 Create Order con tarjeta guardada (token).....	34
17.5 Captura automática con token.....	35
17.6 Propagación del CMID al header del backend	35
18. MSI y MCI	36
18.1 Tarjeta nueva — callbacks del SDK.....	36
18.2 Submit con installments.....	37
18.3 Tarjeta guardada — calculated-financing-options	37
18.4 Create Order con token + installments (REST usa snake_case)	37
19. Vault — Tokenización de Tarjetas.....	39
19.1 customer.id — identificador estable del comprador	39
19.2 Vault-with-Purchase	39
19.3 Listar, cobrar y eliminar tokens	39
19.4 Tarjetas de prueba para Vault (Sandbox).....	40

20. 3DS Risk Initiated y liabilityShift.....	41
20.1 Modalidades.....	41
20.2 Lógica obligatoria en onApprove.....	41
20.3 Tabla de decisión por valor de liabilityShift.....	41
21. Orquestación End-to-End.....	42
21.1 Momento 1 — Inicialización del checkout (una sola vez por sesión).....	42
21.2 Momento 2 — Antes de cada Create Order.....	42
21.3 Momento 3 — Decisión de captura (en onApprove)	42
21.4 Momento 4 — Captura	42
21.5 Regla mnemotécnica	42
22. Caso de Uso Especializado — BOPIS.....	43
23. Puntos de Integración (Mapa de Endpoints).....	44
24. Consideraciones de Seguridad.....	45
24.1 Custodia de credenciales (NFR-02).....	45
24.2 Aislamiento PCI / CSP / Logging.....	45
25. Consideraciones Operativas	46
25.1 Métricas y observabilidad.....	46
25.2 Tolerancia a fallos y promoción a Live	46
26. Estrategia de Pruebas y Tarjetas de Sandbox	48
26.1 Tarjetas para Vault y tokenización (Sandbox).....	48
26.2 Tarjetas para 3DS Risk Initiated.....	48
26.2.1 Flujos Frictionless (sin modal al usuario).....	48
26.2.2 Flujos Step-Up (con modal al usuario).....	49
26.3 Diccionario de valores	50
26.4 Casos de prueba mínimos (TC-01 a TC-14)	50
27. Convenciones de Nombres REST vs SDK	52
28. Asunciones, Dependencias y Restricciones.....	53
28.1 Asunciones	53
28.2 Dependencias externas.....	53
28.3 Restricciones.....	53
29. Riesgos y Mitigaciones.....	55
30. Criterios de Aceptación y Checklist Pre-producción	57
Apéndice A — Diagnóstico de Errores Frecuentes	59
Apéndice B — Limitaciones y Trabajo Futuro.....	61

B.1 Limitaciones conocidas.....	61
B.2 Trabajo futuro sugerido	61

Parte I — Contexto de la solución

1. Resumen Ejecutivo

Este Solution Design Document define la integración de PayPal Advanced Credit and Debit Card (ACDC) en una experiencia de checkout web. La solución habilita el cobro con tarjetas de crédito y débito directamente en el sitio del comercio mediante el componente Card Fields del PayPal JavaScript SDK, integrado con cinco capacidades adicionales: tokenización (Vault), pagos a meses (MSI y MCI), autenticación 3-D Secure en modalidad Risk Initiated, y dos servicios de evaluación de riesgo (Fraudnet y Set Transaction Context).

- Reducción del alcance PCI DSS, manteniendo SAQ A al evitar que el PAN y el CVV toquen la infraestructura del comercio.
- Conversión optimizada mediante cobro nativo en el sitio (sin redirección al checkout de PayPal).
- Protección frente a contracargos mediante Chargeback Protection (CBP) y autenticación 3DS Risk Initiated.
- Reutilización de tarjetas del comprador en compras subsecuentes mediante tokens opacos de Vault.
- Mensualidades con o sin intereses para el comprador en mercados elegibles.
- Evaluación de riesgo pre-transaccional mediante device fingerprinting (Fraudnet) y contexto del comprador (STC).

2. Contexto de Negocio y Drivers

2.1 Drivers de negocio

Driver	Descripción
Conversión y experiencia de usuario	Mantener al comprador dentro del sitio del comercio durante el cobro reduce la fricción. Card Fields permite que la marca, los estilos y la navegación pertenezcan al comercio.
Reducción de costos PCI DSS	El uso de iframes hospedados por PayPal aísla el PAN y el CVV del entorno del comercio, permitiendo cumplimiento PCI bajo SAQ A en lugar de SAQ D.
Protección contra fraude y contracargos	3DS Risk Initiated + Chargeback Protection trasladan al banco emisor la responsabilidad por contracargos fraudulentos cuando 3DS es exitoso.
Aumento del ticket promedio mediante Installments	MSI (costo absorbido por el comercio) e MCI (costo pagado por el comprador) en mercados elegibles incrementan tasas de conversión y ticket promedio.
Recurrencia y retención	Vault permite cobrar a tarjetas guardadas sin re-ingreso, habilitando casos de uso de suscripción y one-click checkout.
Evaluación de riesgo enriquecida	Fraudnet y STC alimentan el motor de riesgo de PayPal con telemetría del navegador y contexto del comprador, mejorando tasas de aprobación y reduciendo falsos positivos.

2.2 Drivers regulatorios

Driver	Descripción
PCI DSS	El estándar exige que el manejo de datos de tarjeta cumpla controles estrictos. Card Fields delega ese manejo a PayPal.
PSD2 / SCA	3DS Risk Initiated proporciona la autenticación reforzada cuando el motor de riesgo o el banco emisor lo demandan.
Regulaciones locales de protección al consumidor	El consentimiento explícito para guardar la tarjeta (Vault-with-Purchase) y la transparencia del cargo de financiamiento (MCI) deben reflejarse en la UI conforme a la normativa del mercado destino.

3. Alcance de la Solución

3.1 Capacidades en alcance

Capacidad	Descripción
Cobro con tarjeta nueva	Captura de datos de tarjeta vía Card Fields (iframes de PayPal) y procesamiento mediante Create Order + Capture Order.
Tokenización (Vault-with-Purchase)	Guardado opcional de la tarjeta al completar la primera compra exitosa.
Listado y eliminación de tokens	Gestión de los métodos de pago guardados de un comprador.
Cobro con tarjeta guardada (token)	Reutilización de un PAYMENT_METHOD_TOKEN para crear órdenes sin re-ingreso de datos.
Installments — MSI	Compras a meses sin intereses para el comprador (costo absorbido por el comercio).
Installments — MCI	Compras a meses con costo de financiamiento al comprador.
3DS Risk Initiated	Autenticación 3-D Secure decidida por el motor de riesgo de PayPal; el comercio solo valida liabilityShift en onApprove.
Fraudnet	Recolección de telemetría de dispositivo y navegador en el cliente.
Set Transaction Context (STC)	Envío de contexto del comprador al motor de riesgo antes de cada Create Order.
BOPIS	Variante del payload de envío para pickup en tienda física.

3.2 Capacidades fuera de alcance

Capacidad	Razón de exclusión
Pago con cuenta PayPal o botón de marca PayPal	La solución se centra exclusivamente en cobro con tarjeta vía Card Fields.
APMs: Pay Later, BNPL, Venmo, billeteras locales	No requeridos por el alcance ACDC.
3DS Merchant Initiated	Aplica a cuentas con Fraud Protection (FP). Este SDD describe el flujo Risk Initiated (CBP).
Suscripciones gestionadas por PayPal Billing Plans	El cobro recurrente mediante token de Vault sí está cubierto; las suscripciones gestionadas por PayPal Billing son una solución distinta.

Capacidad	Razón de exclusión
Webhooks de eventos asíncronos (refunds, disputas)	Requieren un SDD complementario de back-office.
Conciliación contable y reporting financiero	Fuera del alcance del checkout.

4. Stakeholders y Audiencia

Rol	Responsabilidad respecto a este SDD
Solution Architect	Valida la arquitectura, asegura la consistencia con el portafolio del comercio y aprueba el documento.
Product Manager (e-commerce)	Valida que el alcance funcional cubre los requisitos de negocio y la experiencia de checkout deseada.
Integration Engineer (PayPal)	Apoya al comercio en la implementación, valida configuraciones comerciales (CBP, Vault, Installments, STC) y entrega industry packs cuando aplica.
Integration Engineer / Tech Lead (comercio)	Lidera la implementación técnica y es propietario del código del frontend y del backend.
Equipo de Riesgo y Antifraude (comercio)	Valida la configuración de Fraudnet y STC, define el set de campos additional_data y monitorea métricas de fraude.
Equipo de Cumplimiento PCI (comercio)	Verifica que la integración mantiene el alcance SAQ A.
Equipo de QA (comercio)	Ejecuta el plan de pruebas en Sandbox antes de la promoción a Live.
Operaciones (comercio)	Monitorea el checkout en producción, gestiona alertas e incidentes.

5. Glosario de Términos y Acrónimos

Término	Definición
ACDC	Advanced Credit and Debit Card. Producto de PayPal para procesar tarjetas directamente en el sitio del comercio mediante el JavaScript SDK con Card Fields.
Card Fields	Componente del SDK que renderiza los campos de tarjeta como <iframe> hospedados por el dominio de PayPal. El PAN y el CVV nunca llegan al servidor del comercio.
Order	Recurso REST (/v2/checkout/orders) que representa la intención de cobro. Identificado por order_id.
Capture	Operación que ejecuta el cobro real (POST /v2/checkout/orders/{id}/capture). Hasta que ocurre la captura, el dinero no se mueve.
Vault	Servicio de tokenización de PayPal. Almacena el método de pago y devuelve un PAYMENT_METHOD_TOKEN opaco.
MSI	Meses Sin Intereses. El costo del financiamiento lo absorbe el comercio. total_consumer_fee.value === "0.00".
MCI	Meses Con Intereses. El costo lo paga el comprador. total_consumer_fee.value > "0.00". Requiere fee_reference_id.
3DS Risk Initiated	El motor de riesgo de PayPal decide si lanzar 3DS. El comercio no envía instrucciones 3DS; solo valida liabilityShift en onApprove. Requiere CBP.
3DS Merchant Initiated	El comercio decide explícitamente cuándo forzar 3DS. Requiere Fraud Protection (FP). No está en el alcance de este SDD.
liabilityShift	Campo del callback onApprove: undefined (sin desafío), "POSSIBLE" (3DS exitoso), "N" (fallido), "U" (no disponible).
access_token	Token Bearer OAuth2 que el servidor usa para autenticarse con la API REST de PayPal. Nunca debe enviarse al navegador.
id_token	Token JWT que el servidor entrega al navegador para inicializar el SDK. Se asigna al atributo data-sdk-client-token.
CMID	Client Metadata ID. Identificador alfanumérico de hasta 32 caracteres sin guiones, generado una sola vez por sesión de checkout. Actúa como hilo de correlación entre Fraudnet, STC, el SDK y los headers PayPal-Client-Metadata-Id.
Fraudnet	Snippet JavaScript de PayPal que recopila device fingerprinting y señales de comportamiento del navegador. Se compone de dos <script> que deben inyectarse una sola vez por sesión.

Término	Definición
STC	Set Transaction Context. Endpoint PUT /v1/risk/transaction-contexts/{merchant_id}/{cmid}. Envía contexto del comprador a PayPal antes de cada Create Order. Es no bloqueante.
CBP	Chargeback Protection. Producto comercial de PayPal que cubre disputas elegibles. Prerrequisito para 3DS Risk Initiated.
PCI DSS	Payment Card Industry Data Security Standard. Card Fields permite cumplir con SAQ A.
SAQ A	Self-Assessment Questionnaire A. Cuestionario PCI simplificado aplicable cuando el comercio no almacena ni procesa datos de cuenta.
BOPIS	Buy Online, Pickup In Store. El comprador paga en línea pero recoge el pedido en una tienda física. Modifica el objeto shipping de la orden.
NFR	Non-Functional Requirement. Requisito no funcional.

Parte II — Definición de la solución

6. Visión General de la Solución

La solución se compone de tres planos con responsabilidades estrictamente delimitadas y un identificador de correlación (CMID) que atraviesa los tres durante toda la sesión de checkout.

Plano	Responsabilidad	Lo que NO hace
Navegador del comprador	Genera el CMID, inyecta Fraudnet, monta el SDK, renderiza Card Fields, captura interacciones, dispara submit y reacciona al resultado.	Nunca conoce CLIENT_SECRET, access_token ni MERCHANT_ID. Nunca llama directamente a la API REST de PayPal para operaciones de negocio.
Backend del comercio	Custodia las credenciales, obtiene access_token mediante OAuth2, expone endpoints proxy hacia PayPal, inyecta los headers PayPal-Request-Id y PayPal-Client-Metadata-Id, ejecuta STC.	Nunca recibe el PAN ni el CVV. Nunca expone access_token al frontend.
API REST de PayPal	Procesa órdenes, capturas, tokens de Vault, opciones de Installments, contexto de riesgo (STC) y autenticación 3DS.	Es la única fuente de verdad transaccional.

6.1 Componentes de la solución

Componente	Plano	Función
Card Fields	Navegador	Renderiza inputs de tarjeta como iframes seguros.
Fraudnet	Navegador	Telemetría de dispositivo/navegador.
PayPal JavaScript SDK	Navegador	Orquesta Card Fields, callbacks de Installments, ejecución de 3DS.
OAuth2 Service	Backend	Obtiene y cachea access_token; expone id_token al frontend.
Orders Proxy	Backend	Crea y captura órdenes. Inyecta headers de idempotencia y correlación.
Vault Proxy	Backend	Lista y elimina tokens.

Componente	Plano	Función
Installments Proxy	Backend	Consulta opciones de financiamiento para tokens guardados.
STC Proxy	Backend	Envía contexto del comprador a /v1/risk/transaction-contexts.
API REST PayPal	PayPal	Procesamiento real.

6.2 Identificador de correlación: el CMID

El CMID es un identificador alfanumérico de hasta 32 caracteres sin guiones, generado una sola vez por sesión de checkout. Atraviesa los tres planos:

Navegador	Backend del comercio	API PayPal
1. genera CMID		
2. CMID → Fraudnet "f"		
3. CMID → SDK data-client-metadata-id		
4. CMID → body._cmid →	PayPal-Client-Metadata-Id →	/v2/checkout/orders
	PayPal-Client-Metadata-Id →	/v2/checkout/orders/{id}/capture
		→ /v1/risk/transaction-contexts/{mid}/{cmid}

⚠ Sin un CMID consistente entre los tres planos, PayPal Risk no puede correlacionar las señales de Fraudnet, el contexto de STC y la transacción real.

7. Requisitos Funcionales

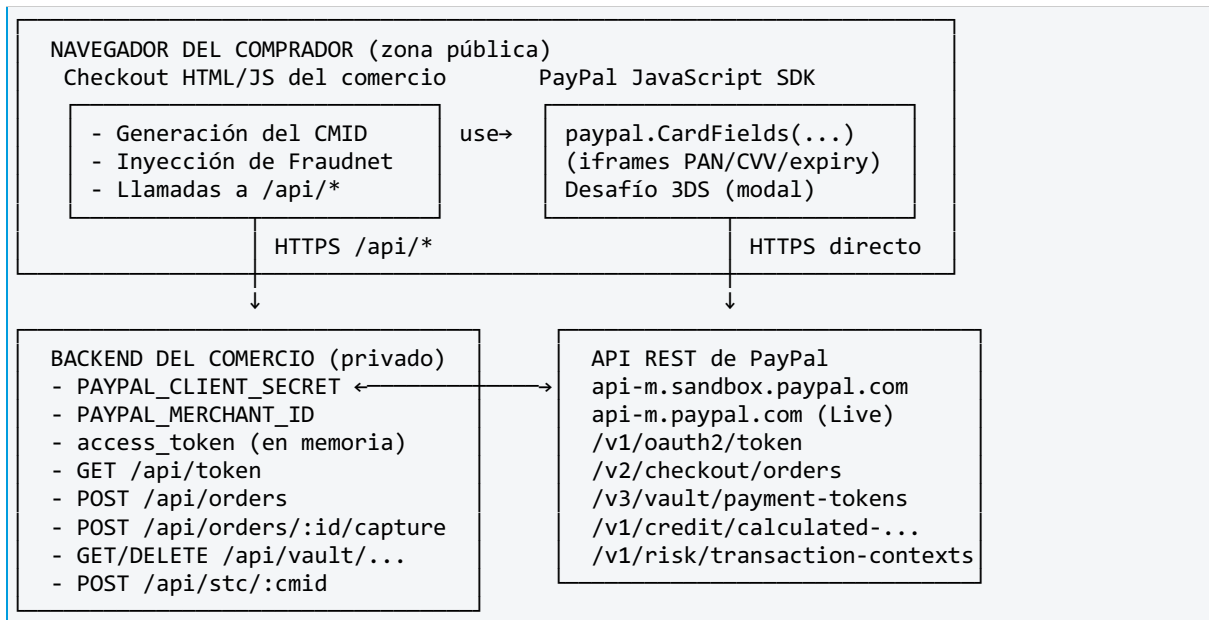
ID	Requisito	Prioridad
RF-01	El sistema debe permitir al comprador ingresar los datos de su tarjeta sin que estos viajen al backend del comercio.	Obligatorio
RF-02	El sistema debe crear una orden con intent: "CAPTURE" y un breakdown matemáticamente consistente ($\text{item_total} + \text{tax_total} + \text{shipping} - \text{discount} = \text{amount.value}$).	Obligatorio
RF-03	El sistema debe ejecutar la captura del pago únicamente cuando el resultado de 3DS sea favorable (liabilityShift ausente o igual a "POSSIBLE").	Obligatorio
RF-04	El sistema debe ofrecer al comprador la opción de guardar su tarjeta para futuras compras (Vault-with-Purchase) mediante consentimiento explícito.	Obligatorio
RF-05	El sistema debe listar las tarjetas guardadas de un comprador autenticado y permitirle pagar con un token sin re-ingresar los datos.	Obligatorio
RF-06	El sistema debe permitir al comprador eliminar una tarjeta guardada de su perfil.	Obligatorio
RF-07	El sistema debe presentar al comprador las opciones de financiamiento a meses (Installments) elegibles, distinguiendo MSI y MCI con etiquetas claras.	Obligatorio
RF-08	El sistema debe transmitir a PayPal Risk el contexto del comprador antes de cada Create Order, sin bloquear el checkout en caso de error.	Obligatorio
RF-09	El sistema debe inyectar Fraudnet en el navegador una sola vez por sesión de checkout, con el CMID generado al inicio.	Obligatorio
RF-10	El sistema debe mostrar al comprador un mensaje claro y accionable cuando 3DS falle, sin capturar el pago.	Obligatorio
RF-11	El sistema debe soportar BOPIS sustituyendo el objeto shipping con la dirección del punto de pickup.	Opcional
RF-12	El sistema debe permitir reintentos del comprador dentro de una misma sesión sin regenerar el CMID ni recargar Fraudnet.	Obligatorio

8. Requisitos No Funcionales

ID	Categoría	Requisito
NFR-01	Seguridad — PCI DSS	El PAN y el CVV nunca deben tocar la infraestructura del comercio. La integración debe permitir cumplimiento bajo SAQ A.
NFR-02	Seguridad — secretos	PAYPAL_CLIENT_SECRET, PAYPAL_MERCHANT_ID y access_token nunca deben transmitirse al navegador, ni almacenarse en código fuente versionado, ni aparecer en logs.
NFR-03	Seguridad — transporte	Todas las comunicaciones deben usar TLS 1.2 o superior.
NFR-04	Seguridad — CSP	El sitio debe declarar una CSP que liste explícitamente los orígenes de PayPal (www.paypal.com, www.paypalobjects.com, c.paypal.com, api-m.paypal.com).
NFR-05	Disponibilidad	La caída de servicios accesorios (STC, Fraudnet) no debe interrumpir el flujo de pago. Su comportamiento es no bloqueante.
NFR-06	Idempotencia	El sistema debe garantizar que reintentos de Create Order o Capture Order no generen duplicados mediante PayPal-Request-Id.
NFR-07	Latencia (p95)	El tiempo total entre submit y respuesta al usuario no debe exceder los SLOs internos del comercio.
NFR-08	Trazabilidad	El sistema debe loguear liabilityShift, enrollment_status, authentication_status, order_id, capture_id, decisión de captura y código de error, sin loguear PAN ni CVV.
NFR-09	Internacionalización	El SDK debe cargarse con el locale correspondiente al mercado destino.
NFR-10	Accesibilidad	La UI de Card Fields debe cumplir WCAG 2.1 AA.
NFR-11	Escalabilidad	El backend debe cachear el access_token hasta el 90% de expires_in.
NFR-12	Compatibilidad de navegador	La integración debe validar cardField.isEligible() antes de renderizar.
NFR-13	Auditabilidad	Cada transacción debe ser reconstruible a partir de los logs: CMID, PayPal-Request-Id, order_id, invoice_id, custom_id.

9. Arquitectura de la Solución

9.1 Diagrama lógico



9.2 Reglas no negociables de la arquitectura

1. `PAYPAL_CLIENT_SECRET` reside únicamente en variables de entorno del backend. Jamás se versiona, jamás se envía al navegador.
2. El navegador nunca llama directamente a `api-m.paypal.com` para operaciones de negocio.
3. El navegador nunca recibe el `access_token`. Recibe solo `clientId` (público) e `id_token`.
4. El PAN y el CVV nunca tocan el backend del comercio. Card Fields los aísla en `<iframe>` del dominio de PayPal.
5. Los datos de carrito, comprador y dirección de envío nunca son hardcoded en el código del frontend.

9.3 Frontera de seguridad por asset

Asset	Frontend	Backend	API PayPal
CLIENT_ID	Sí	Sí	—
CLIENT_SECRET	No	Sí (env)	—
MERCHANT_ID	No	Sí (env)	—
access_token	No	Sí (memoria)	—
id_token	Sí (al SDK)	Sí	—
PAN / CVV	iframe PayPal	No	Sí
PAYMENT_METHOD_TOKEN	Sí (referencia)	Sí	Sí
CMID	Sí (genera)	Sí (recibe en body)	Sí (header)

10. Prerrequisitos y Configuración del Entorno

10.1 Habilitaciones comerciales requeridas

Capacidad	Aplica a
ACDC con Card Fields	Procesamiento de tarjetas mediante el SDK.
Vault v3	Tokenización de métodos de pago.
Installments (Buyer / Seller financed)	Cobro a meses (MSI y MCI) en el mercado destino.
Chargeback Protection (CBP)	Activación de 3DS Risk Initiated.
Fraudnet + Set Transaction Context	Acceso al endpoint /v1/risk/transaction-contexts.

10.2 Credenciales y configuración del backend

Variable de entorno	Origen	Visibilidad
PAYPAL_CLIENT_ID	PayPal Developer Dashboard → app del comercio.	Pública (puede ir al frontend).
PAYPAL_CLIENT_SECRET	PayPal Developer Dashboard → app del comercio.	Secreta — solo backend.
PAYPAL_MERCHANT_ID	Perfil de la cuenta merchant en PayPal.	Privada — solo backend (requerida para STC).
PAYPAL_API_BASE	https://api-m.sandbox.paypal.com (Sandbox) · https://api-m.paypal.com (Live).	Privada.

 Las credenciales de Sandbox y Live son distintas. PAYPAL_API_BASE, PAYPAL_CLIENT_ID, PAYPAL_CLIENT_SECRET y PAYPAL_MERCHANT_ID deben corresponder al mismo entorno. Una mezcla produce 401 Unauthorized.

10.3 Requisitos del entorno

- TLS / HTTPS obligatorio en producción. Card Fields y Fraudnet no operan sobre HTTP.
- Content Security Policy (CSP): script-src <https://www.paypal.com> <https://www.paypalobjects.com> <https://c.paypal.com>; frame-src

<https://www.paypal.com>; connect-src <https://api-m.paypal.com> <https://api-m.sandbox.paypal.com>

- Soporte de Web Crypto API en el navegador (para `crypto.randomUUID()`). Para soportar navegadores legacy, implementar fallback.

Parte III — Diseño detallado

11. Autenticación OAuth2 y Emisión del id_token

El backend del comercio es el único componente autorizado para hablar OAuth2 con PayPal. El propósito de este paso es obtener simultáneamente: un access_token para que el backend llame a la API REST de PayPal, y un id_token que el backend reenvía al navegador para inicializar el SDK.

11.1 Request

```
POST {PAYPAL_API_BASE}/v1/oauth2/token
Authorization: Basic <BASE64(CLIENT_ID:CLIENT_SECRET)>
Content-Type: application/x-www-form-urlencoded

grant_type=client_credentials&response_type=id_token
```

 response_type=id_token debe estar presente. Sin él, la respuesta no incluye el id_token necesario para el SDK del frontend.

11.2 Response

```
{
  "access_token": "<ACCESS_TOKEN>",
  "id_token": "<ID_TOKEN>",
  "expires_in": 32400,
  "token_type": "Bearer"
}
```

11.3 Caché del access_token

Práctica	Justificación
Cachear el access_token en memoria del proceso del backend hasta el 90% de expires_in.	Evita llamadas innecesarias a /v1/oauth2/token y reduce latencia en el camino crítico de Create Order.
No persistir el access_token en disco, base de datos ni logs.	Token Bearer con poder transaccional; su exposición compromete la cuenta.
Refrescar inmediatamente ante un 401 y reintentar una vez.	Cubre el edge case de token revocado por rotación de credenciales.

11.4 Endpoint expuesto al frontend

```
GET /api/token
→ 200 OK
{
  "clientId": "<PAYPAL_CLIENT_ID>",
  "idToken": "<ID_TOKEN>"
}
```

⚠ Este endpoint nunca debe devolver el `access_token`. Si por error se entrega al navegador, cualquier visitante puede operar la cuenta de PayPal del comercio.

11.5 Implementación de referencia (Node.js)

```
async function generateAccessToken() {
  const credentials = Buffer.from(
    `${process.env.PAYPAL_CLIENT_ID}:${process.env.PAYPAL_CLIENT_SECRET}`
  ).toString("base64");
  const response = await fetch(`${process.env.PAYPAL_API_BASE}/v1/oauth2/token`, {
    method: "POST",
    headers: { "Authorization": `Basic ${credentials}`,
      "Content-Type": "application/x-www-form-urlencoded" },
    body: "grant_type=client_credentials&response_type=id_token"
  });
  if (!response.ok) throw new Error(`OAuth failed: ${response.status}`);
  return response.json(); // { access_token, id_token, expires_in, ... }
}
```

12. Generación del Client Metadata ID (CMID)

El CMID es un identificador único alfanumérico de hasta 32 caracteres sin guiones que actúa como hilo de correlación entre: el campo "f" de Fraudnet, el atributo data-client-metadata-id del SDK, la URL del endpoint de STC, y el header PayPal-Client-Metadata-Id de Create Order y Capture Order.

12.1 Reglas de ciclo de vida

Regla	Detalle
Único por sesión de checkout	Se genera una sola vez al inicializar la página del checkout.
Persistente entre reintentos	Si el comprador falla un pago y reintenta dentro de la misma sesión, el CMID no se regenera. Tampoco se recarga Fraudnet.
Persistente entre métodos de pago	Si el comprador alterna entre tarjeta nueva y tarjeta guardada en la misma sesión, el CMID se conserva.
Se regenera en transacción nueva	Solo después de que la transacción anterior fue completada o cancelada definitivamente se genera un nuevo CMID y se recarga Fraudnet.
Se propaga server-side por el backend	El frontend lo entrega al backend en el body de Create Order/Capture Order; el backend lo reenvía como header HTTP a PayPal.

12.2 Implementación de referencia

```
function generateCMID() {
  if (typeof crypto !== "undefined" && typeof crypto.randomUUID === "function") {
    return crypto.randomUUID().replace(/-/g, ""); // 32 caracteres hex
  }
  // Fallback para entornos sin Web Crypto API
  return "xxxxxxxx4xxxymxxxxxxxxxxxxx".replace(/[xy]/g, (c) => {
    const r = (Math.random() * 16) | 0;
    return (c === "x" ? r : (r & 0x3) | 0x8).toString(16);
  });
}
const cmid = generateCMID(); // ej. "a1b2c3d4e5f6a1b2c3d4e5f6a1b2c3d4"
```

12.3 Validación

```
const isValidCMID = (cmid) =>
  typeof cmid === "string" && cmid.length >= 1 && cmid.length <= 32 &&
  /^[0-9a-zA-Z]+$/.test(cmid);
```


13. Integración de Fraudnet

Fraudnet es el primer servicio de riesgo que se inicializa en el navegador, inmediatamente después de generar el CMID y antes de cualquier llamada al backend o al SDK. Recopila device fingerprinting, señales de comportamiento del navegador y telemetría de sesión que PayPal Risk usa para evaluar la transacción.

13.1 Script de configuración (JSON)

```
<script type="application/json"
  fncls="fnparams-dede7cc5-15fd-4c75-a9f4-36c430ee3a99">
{
  "f": "<CMID>",
  "s": "<NOMBRE_CORTO_DEL_MERCHANT>_<MERCHANT_ID>_ACDC"
}
</script>
```

13.2 Script de la librería

```
<script type="text/javascript" src="https://c.paypal.com/da/r/fb.js"></script>
```

 El orden importa. El script JSON de configuración debe estar presente en el DOM antes de que fb.js se ejecute.

13.3 Implementación dinámica

```
let fraudnetLoaded = false;
function loadFraudnet({ cmid, merchantShortName, merchantId }) {
  if (fraudnetLoaded) return;
  fraudnetLoaded = true;
  const config = document.createElement("script");
  config.type = "application/json";
  config.setAttribute("fncls", "fnparams-dede7cc5-15fd-4c75-a9f4-36c430ee3a99");
  config.textContent = JSON.stringify({ f: cmid, s:
`${merchantShortName}_${merchantId}_ACDC` });
  document.body.appendChild(config);
  const lib = document.createElement("script");
  lib.type = "text/javascript";
  lib.src = "https://c.paypal.com/da/r/fb.js";
  document.body.appendChild(lib);
}
```

13.4 Reglas operativas

Regla	Detalle
Una sola carga por sesión de checkout.	Repetir la inyección invalida la sesión de Fraudnet y produce señales inconsistentes.
Cubre nueva tarjeta y tarjeta guardada.	La misma sesión de Fraudnet aplica a ambos flujos; no recargar al cambiar de método de pago.
No recargar tras un intento fallido.	El reintento dentro de la misma sesión debe usar la misma carga de Fraudnet y el mismo CMID.
Recargar solo en transacción nueva.	Después de una transacción completada o cancelada definitivamente, generar nuevo CMID y volver a invocar loadFraudnet.
CSP-friendly.	El origen <code>https://c.paypal.com</code> debe estar en script-src de la Content Security Policy del sitio.

14. Set Transaction Context (STC)

STC permite al comercio enviar contexto del comprador a PayPal Risk antes de cada Create Order. La operación es no bloqueante: cualquier error debe loguearse pero nunca detener el flujo de checkout.

14.1 Endpoint

```
PUT {PAYPAL_API_BASE}/v1/risk/transaction-contexts/<MERCHANT_ID>/<CMID>
Authorization: Bearer <ACCESS_TOKEN>
Content-Type: application/json
```

14.2 Body — set genérico para Retail

```
{
  "additional_data": [
    { "key": "sender_account_id", "value": "<ID_DEL_COMPRADOR_EN_LA_PLATAFORMA>" },
    { "key": "sender_first_name", "value": "<NOMBRE_DEL_COMPRADOR>" },
    { "key": "sender_last_name", "value": "<APELLIDO_DEL_COMPRADOR>" },
    { "key": "sender_email", "value": "<EMAIL_DEL_COMPRADOR>" },
    { "key": "sender_phone", "value": "<TELEFONO_SOLO_DIGITOS>" },
    { "key": "sender_country_code", "value": "<PAIS_ISO_ALPHA2>" },
    { "key": "sender_create_date", "value": "<FECHA_DE_ALTA_DEL_USUARIO>" },
    { "key": "highrisk_txn_flag", "value": 0 },
    { "key": "vertical", "value": "<VERTICAL_DEL_NEGOCIO>" },
    { "key": "cd_string_one", "value": "<NIVEL_DE_CONFIANZA_DEL_USUARIO>" }
  ]
}
```

14.3 Referencia de campos

Campo	Tipo	Descripción	Valores aceptados
sender_account_id	string	ID único del comprador en la plataforma del comercio.	Alfanumérico estable entre sesiones.
sender_first_name	string	Nombre registrado del comprador.	Alfanumérico.
sender_last_name	string	Apellido registrado del comprador.	Alfanumérico.
sender_email	string	Email validado del comprador.	Formato RFC 5322.

Campo	Tipo	Descripción	Valores aceptados
sender_phone	string	Teléfono del comprador, solo dígitos.	[0-9]+
sender_country_code	string	País del comprador en ISO 3166-1 Alpha-2.	MX, US, BR, etc.
sender_create_date	string	Fecha de alta del usuario en la plataforma del comercio.	yyyy-mm-ddThh:mm:ssZ yyyy-mm-dd yyyyymmdd
highrisk_txn_flag	number	¿Es una transacción de alto riesgo?	0 = normal, 1 = alto riesgo. No es string.
vertical	string	Vertical del negocio.	Retail, Travel, Gaming, etc. (consultar industry pack con el IE).
cd_string_one	string	Nivel de confianza del comprador asignado por la plataforma.	"1" confiable / "0" desconocido / "2" no confiable.

 Industry packs: El set anterior es el genérico para Retail. Verticales como Travel, OTAs, Financial Services, Gaming y plataformas reguladas requieren campos adicionales. Solicitar el industry pack al Integration Engineer.

14.4 Manejo de respuesta — comportamiento no bloqueante

Status	Significado	Acción
200	OK. Contexto registrado.	Continuar con Create Order.
400	Body inválido.	Logear el error con detalle, continuar con Create Order.
401	Sin permisos o token expirado.	Refrescar token, logear, continuar con Create Order.
5xx	Error interno de PayPal.	Logear, continuar con Create Order.

 STC nunca puede bloquear el checkout. Una falla de STC reduce la calidad de la evaluación de riesgo pero no impide procesar la transacción.

14.5 Implementación de referencia

```
// Backend (Node.js)
app.post("/api/stc/:cmid", async (req, res) => {
  try {
    const { cmid } = req.params;
    const { access_token } = await generateAccessToken();
    const response = await fetch(
      `${process.env.PAYPAL_API_BASE}/v1/risk/transaction-
      contexts/${process.env.PAYPAL_MERCHANT_ID}/${cmid}`,
      { method: "PUT", headers: { "Authorization": `Bearer ${access_token}`, "Content-
      Type": "application/json" },
        body: JSON.stringify({ additional_data: buildAdditionalDataFromSession(req) }) }
    );
    res.status(200).json({ success: response.ok, status: response.status });
  } catch (err) {
    console.warn("STC failed (non-blocking):", err);
    res.status(200).json({ success: false });
  }
});

// Frontend
async function callSTC(cmId) {
  try { await fetch(`/api/stc/${cmId}`, { method: "POST" }); }
  catch (err) { console.warn("STC could not complete (non-blocking):", err); }
}
```

15. Carga del JavaScript SDK de PayPal

Con el CMID generado, Fraudnet inyectado y el id_token obtenido del backend, el siguiente paso es cargar el SDK. La carga debe ocurrir después de §12, §13 y la obtención de clientId + idToken.

15.1 Construcción de la URL del SDK

```
const url = new URL("https://www.paypal.com/sdk/js");
url.searchParams.set("client-id", clientId);
url.searchParams.set("currency", "<MONEDA_ISO_4217>");
url.searchParams.set("locale", "<LOCALE_BCP47>");
url.searchParams.set("components", "card-fields"); // únicamente card-fields

const script = document.createElement("script");
script.src = url.toString();
script.setAttribute("data-sdk-client-token", idToken);
script.setAttribute("data-client-metadata-id", cmid); // mismo CMID de Fraudnet y STC
script.onload = () => { renderCardFields(); };
document.body.appendChild(script);
```

 El data-client-metadata-id debe ser exactamente el mismo valor que se usó en el campo "f" de Fraudnet (§13) y que se usará en la URL de STC (§14) y en el header PayPal-Client-Metadata-Id. Cualquier divergencia rompe la correlación de riesgo.

16. Estructura HTML y Estilos de Card Fields

16.1 Contenedores del DOM

```
<div id="card-number-field-container"></div>
<div id="card-expiry-field-container"></div>
<div id="card-cvv-field-container"></div>
<div id="card-name-field-container"></div>
<select id="installments-select"><option value="">Sin meses</option></select>
<label><input type="checkbox" id="vault" /> Guardar tarjeta para futuras compras</label>
<button id="card-field-submit" type="button">Pagar</button>
<div id="payment-result" role="status" aria-live="polite"></div>
```

16.2 Renderizado

```
function renderCardFields() {
  const cardField = paypal.CardFields({
    style: styleObject,
    createOrder: createOrderFn,
    onApprove: onApproveFn,
    onError: onErrorFn,
    installments: installmentsBlock
  });
  if (!cardField.isEligible()) return;
  cardField.NumberField().render("#card-number-field-container");
  cardField.ExpiryField().render("#card-expiry-field-container");
  cardField.CVVField().render("#card-cvv-field-container");
  cardField.NameField().render("#card-name-field-container");
}
```

16.3 Estilos — dos capas independientes

Capa 1 — CSS del contenedor (tu control total): el <div> que aloja cada iframe vive en el dominio del comercio. Controla dimensiones, borde y layout.

```
<link rel="stylesheet"
href="https://www.paypalobjects.com/webstatic/en_US/developer/docs/css/cardfields.css">
```

Capa 2 — styleObject del SDK (estilos dentro del iframe): el <input> real donde el usuario escribe está dentro del <iframe> de PayPal. La única vía para estilizarlo es el objeto style que se pasa al constructor de paypal.CardFields.

```
const styleObject = {
  input: { "font-size": "14px", "font-family": "system-ui, sans-serif", "color":
"<COLOR_TEXTO_NORMAL>" },
  ".invalid": { "color": "<COLOR_TEXTO_INVALIDO>" },
  ":focus": { "color": "<COLOR_TEXTO_FOCUS>" },
  ":hover": { "color": "<COLOR_TEXTO_HOVER>" }
};
```

💡 Accesibilidad (NFR-10): No usar color como único indicador de invalidez. Acompañar siempre con texto y/o icono para usuarios con baja visión o daltonismo.

17. Creación y Captura de Órdenes

17.1 Headers HTTP obligatorios

Header	Create	Capture	Descripción
Authorization: Bearer <ACCESS_TOKEN>	Sí	Sí	Token Bearer obtenido en §11.
Content-Type: application/json	Sí	Sí	El body es JSON.
PayPal-Request-Id: <UUID>	Sí	Sí	Idempotencia. El mismo UUID debe usarse para Create Order y para su Capture asociado. Generar uno nuevo por transacción nueva.
PayPal-Client-Metadata-Id: <CMID>	Sí	Sí	Vincula la transacción con Fraudnet y STC. Mismo CMID que en "f" de Fraudnet y en la URL de STC.

17.2 Create Order — payload completo (tarjeta nueva con Vault)

```
{
  "intent": "CAPTURE",
  "application_context": {
    "brand_name": "<NOMBRE_VISIBLE_DEL_COMERCIO>", "locale": "<LOCALE_BCP47>",
    "shipping_preference": "SET_PROVIDED_ADDRESS", "user_action": "PAY_NOW",
    "return_url": "<URL_DE_RETORNO_REAL_DEL_COMERCIO>",
    "cancel_url": "<URL_DE_CANCELACION_REAL_DEL_COMERCIO>"
  },
  "payer": {
    "email_address": "<EMAIL_DEL_COMPRADOR>",
    "name": { "given_name": "<NOMBRE>", "surname": "<APELLIDO>" },
    "phone": { "phone_type": "MOBILE", "phone_number": { "national_number":
"<TELEFONO_SOLO_DIGITOS>" } }
  },
  "purchase_units": [{
    "invoice_id": "<INVOICE_ID_UNICO_DEL_COMERCIO>", "custom_id": "<ORDER_ID_INTERNO>",
    "description": "<DESCRIPCION_BREVE_DEL_PEDIDO>",
    "amount": {
      "currency_code": "<MONEDA_ISO_4217>", "value": "<TOTAL_DEL_CARRITO>",
      "breakdown": {
        "item_total": { "currency_code": "<MONEDA>", "value": "<SUMA_UNIT_AMOUNT_X_QTY>"
      },
      "tax_total": { "currency_code": "<MONEDA>", "value": "<SUMA_TAX_X_QTY>" },
      "shipping": { "currency_code": "<MONEDA>", "value": "<COSTO_DE_ENVIO>" },
      "discount": { "currency_code": "<MONEDA>", "value": "<DESCUENTO_APLICADO>" }
    }
  }],
  "items": [{
    "name": "<NOMBRE_DEL_PRODUCTO>", "description": "<DESCRIPCION>", "sku": "<SKU>",
    "quantity": "<CANTIDAD>",
```

```

    "unit_amount": { "currency_code": "<MONEDA>", "value":
"<PRECIO_UNITARIO_SIN_IMPUESTOS>" },
    "tax": { "currency_code": "<MONEDA>", "value": "<IMPUESTO_POR_UNIDAD>" },
    "category": "PHYSICAL_GOODS"
  }],
  "shipping": {
    "name": { "full_name": "<NOMBRE_COMPLETO_DEL_DESTINATARIO>" },
    "address": {
      "address_line_1": "<CALLE_Y_NUMERO>", "address_line_2": "<COLONIA_O_REFERENCIA>",
      "admin_area_2": "<CIUDAD_O_MUNICIPIO>", "admin_area_1": "<ESTADO_CODIGO>",
      "postal_code": "<CODIGO_POSTAL>", "country_code": "<PAIS_ISO_ALPHA2>"
    }
  }
}],
"payment_source": { "card": { "attributes": {
  "customer": { "id": "<ID_DEL_USUARIO_AUTENTICADO_DEL_COMERCIO>" },
  "vault": {
    "store_in_vault": "ON_SUCCESS", "usage_type": "MERCHANT",
    "customer_type": "CONSUMER", "permit_multiple_payment_tokens": true
  }
}
}}}
}

```

17.3 Reglas de validación del breakdown

```

amount.value === item_total + tax_total + shipping - discount
item_total    === Σ (item.unit_amount × item.quantity) por línea
tax_total     === Σ (item.tax × item.quantity) por línea

```

Si los valores no cuadran, PayPal responde 422 UNPROCESSABLE_ENTITY con detalle del campo inconsistente.

17.4 Create Order con tarjeta guardada (token)

El payload base es idéntico al de §17.2. Cambia únicamente el payment_source:

```

{
  "payment_source": {
    "token": {
      "id": "<PAYMENT_METHOD_TOKEN_DEL_VAULT>",
      "type": "PAYMENT_METHOD_TOKEN"
    }
  }
}

```

17.5 Captura automática con token

status devuelto	Acción
COMPLETED	La orden se auto-capturó. No llamar a /capture.
APPROVED	Captura manual requerida. Llamar a POST /v2/checkout/orders/{id}/capture.

17.6 Propagación del CMID al header del backend

```
// Frontend – dentro del callback createOrder del SDK
async function createOrderFn() {
  await callSTC(cmId); // §14
  const body = buildOrderPayload(); // §17.2
  body._cmid = cmId; // campo de control interno
  const response = await fetch("/api/orders", {
    method: "POST", headers: { "Content-Type": "application/json" },
    body: JSON.stringify(body)
  });
  return (await response.json()).id;
}

// Backend – ruta POST /api/orders
app.post("/api/orders", async (req, res) => {
  const { _cmid, ...orderPayload } = req.body;
  const { access_token } = await generateAccessToken();
  const requestId = req.headers["x-request-id"] || generateRequestId();
  const response = await fetch(`${process.env.PAYPAL_API_BASE}/v2/checkout/orders`, {
    method: "POST",
    headers: { "Authorization": `Bearer ${access_token}`, "Content-Type":
"application/json",
      "PayPal-Request-Id": requestId, "PayPal-Client-Metadata-Id": _cmid },
    body: JSON.stringify(orderPayload)
  });
  res.status(response.status).json(await response.json());
});
```

18. MSI y MCI

 Clasificación correcta: No deducir MSI/MCI del número de meses. Siempre verificar `total_consumer_fee.value`.

Modalidad	Quién paga el cargo	<code>total_consumer_fee.value</code>	<code>fee_reference_id</code>	Etiqueta de UI
MSI (Meses Sin Intereses)	Comercio (absorbe el costo)	"0.00"	Generalmente opcional	"X meses sin intereses"
MCI (Meses Con Intereses)	Comprador	> "0.00"	Obligatorio	"X meses — cargo \$Y <MONEDA>"

18.1 Tarjeta nueva — callbacks del SDK

```
const cardField = paypal.CardFields({
  installments: {
    onInstallmentsRequested: () => ({
      financingCountryCode: "<PAIS_ISO_ALPHA2>",
      amount: "<TOTAL_DEL_CARRITO>",
      currencyCode: "<MONEDA_ISO_4217>",
      billingCountryCode: "<PAIS_DE_FACTURACION_ISO_ALPHA2>",
      includeBuyerInstallments: true
    }),
    onInstallmentsAvailable: ({ financing_options }) => {
      const opciones = (financing_options || [])
        .filter((o) => o.product === "CARD_ISSUER_INSTALLMENTS")
        .flatMap((o) => o.qualifying_financing_options);
      opciones.forEach((opt) => {
        const isMSI = parseFloat(opt.total_consumer_fee.value) === 0;
        const label = opt.credit_financing.term === 1 ? "Pago en una sola exhibición"
          : isMSI ? `${opt.credit_financing.term} meses sin intereses`
            : `${opt.credit_financing.term} meses - cargo ${opt.total_consumer_fee.value}
              ${opt.total_consumer_fee.currency_code}`;
        // Guardar { term, intervalDuration: opt.credit_financing.interval_duration,
        //           feeReferenceId: opt.fee_reference_id }
      });
    },
    onInstallmentsError: (err) => { /* log + UI de fallback */ }
  }
});
```

18.2 Submit con installments

```
async function submitWithSelectedInstallments(cardField) {
  const selected = readSelectedOptionFromUI();
  if (selected.term === 1) { await cardField.submit({}); return; }
  await cardField.submit({
    installments: {
      term: selected.term,
      intervalDuration: selected.intervalDuration, // "P1M" = mensual
      feeReferenceId: selected.feeReferenceId      // OBLIGATORIO en MCI
    }
  });
}
```

18.3 Tarjeta guardada — calculated-financing-options

```
{
  "financing_country_code": "<PAIS_ISO_ALPHA2>",
  "transaction_amount": { "value": "<TOTAL>", "currency_code": "<MONEDA>" },
  "funding_instrument": {
    "type": "TOKEN",
    "token": { "type": "PAYMENT_METHOD_TOKEN", "payment_method_token": "<TOKEN_DEL_VAULT>"
  }
},
  "flow_context": { "attributes": ["FEE_POLICY_CHARGE_CONSUMER"] }
}
```

 **FEE_POLICY_CHARGE_CONSUMER** es necesario para que la respuesta incluya opciones MCI. Sin él, la respuesta puede contener únicamente MSI.

18.4 Create Order con token + installments (REST usa snake_case)

```
{
  "payment_source": {
    "token": {
      "id": "<PAYMENT_METHOD_TOKEN>", "type": "PAYMENT_METHOD_TOKEN",
      "attributes": {
        "installments": {
          "term": "<TERM_DE_MESES>",
          "interval_duration": "<INTERVAL_DURATION_ISO_8601>",
          "fee_reference_id": "<FEE_REFERENCE_ID_DE_LA_OPCION>"
        }
      }
    }
  }
}
```

Contexto	Formato	Ejemplo
Callback del SDK (cardField.submit)	camelCase	intervalDuration, feeReferenceId
API REST (payment_source.token.attributes.installments)	snake_case	interval_duration, fee_reference_id

 Mezclar formatos es la causa más frecuente de errores UNPROCESSABLE_ENTITY en Installments.

19. Vault — Tokenización de Tarjetas

Vault almacena el método de pago de forma segura y devuelve un `PAYMENT_METHOD_TOKEN` opaco que el comercio reutiliza para cobrar en futuras transacciones sin que el comprador re-ingrese sus datos.

19.1 customer.id — identificador estable del comprador

- Estable: el mismo identificador entre sesiones del usuario.
- Determinista: generable a partir del registro del usuario (no aleatorio por sesión).
- Único por comprador: dos compradores distintos jamás comparten `customer.id`.

19.2 Vault-with-Purchase

Atributo	Valor	Significado
<code>store_in_vault</code>	<code>ON_SUCCESS</code>	Solo guarda la tarjeta si la transacción se completa exitosamente. Tarjetas fallidas no se guardan.
<code>usage_type</code>	<code>MERCHANT</code>	Cobros futuros iniciados por el comercio.
<code>customer_type</code>	<code>CONSUMER</code>	Persona física (vs <code>BUSINESS</code>).
<code>permit_multiple_payment_tokens</code>	<code>true</code>	Permite múltiples tokens para el mismo <code>customer.id</code> (varias tarjetas).

19.3 Listar, cobrar y eliminar tokens

```
// Listar
GET {PAYPAL_API_BASE}/v3/vault/payment-tokens?customer_id=<CUSTOMER_ID>
// Eliminar
DELETE {PAYPAL_API_BASE}/v3/vault/payment-tokens/<PAYMENT_METHOD_TOKEN>
```

💡 Para la UI, los campos útiles de la respuesta son `brand`, `last_digits` y `expiry` para mostrar líneas como "VISA1234 (exp. 03/27)".

19.4 Tarjetas de prueba para Vault (Sandbox)

Marca	Número	Notas
Visa (Bancomer)	4772129056533503	Tarjeta de crédito emitida por Bancomer (México).
Mastercard	5288775404117508	Tarjeta de crédito.
Amex	376680816376961	CVV de 4 dígitos.

20. 3DS Risk Initiated y liabilityShift

3-D Secure (3DS) es un protocolo donde el banco emisor del comprador verifica su identidad. Cuando 3DS es exitoso, la responsabilidad por contracargos fraudulentos se transfiere del comercio al banco emisor.

20.1 Modalidades

Modalidad	Quién decide cuándo lanzar 3DS	Producto requerido	Cobertura
Risk Initiated	Motor de riesgo de PayPal	Chargeback Protection (CBP)	Cubierta en este SDD.
Merchant Initiated	El comercio	Fraud Protection (FP)	No cubierta en este SDD.

20.2 Lógica obligatoria en onApprove

```

async function onApprove(data) {
  if (!data.liabilityShift) {
    // Sin desafío 3DS – el motor de riesgo no lo consideró necesario.
    await capture(data.orderID);
    return;
  }
  if (data.liabilityShift === "POSSIBLE") {
    // 3DS exitoso – el banco emisor asume responsabilidad de contracargo.
    await capture(data.orderID);
    return;
  }
  // liabilityShift presente con cualquier otro valor → no capturar.
  showError("La autenticación no se completó. Intente con otro método de pago.");
}

```

20.3 Tabla de decisión por valor de liabilityShift

Valor	Significado	Acción
undefined (ausente)	No hubo desafío 3DS.	Capturar.
"POSSIBLE"	3DS exitoso. Shift al emisor.	Capturar.
"N"	Autenticación fallida.	No capturar. Mostrar error.
"U" / otros	No disponible o rechazada.	No capturar. Mostrar error.

 No forzar 3DS desde el backend bajo el modelo Risk Initiated. Mezclar Risk Initiated con instrucciones de Merchant Initiated produce comportamientos indefinidos.

Parte IV — Integración y operación

21. Orquestación End-to-End

21.1 Momento 1 — Inicialización del checkout (una sola vez por sesión)

```
1. cmid = generateCMID() [§12]
2. loadFraudnet({ cmid, merchantShortName, merchantId }) [§13]
3. { clientId, idToken } = GET /api/token [§11]
4. (Opcional) Listar tokens guardados [§19.3]
5. Cargar SDK con data-client-metadata-id = cmid [§15]
6. Renderizar Card Fields [§16, §18, §20]
```

21.2 Momento 2 — Antes de cada Create Order

```
1. await callSTC(cmid) [§14]
2. Construir payload de Create Order [§17.2 o §17.4]
3. POST /api/orders + body._cmid = cmid [§17.6]
   → backend inyecta:
      PayPal-Request-Id: <UUID_DE_TRANSACCION>
      PayPal-Client-Metadata-Id: <CMID>
```

21.3 Momento 3 — Decisión de captura (en onApprove)

```
if (!data.liabilityShift) → capturar [§20]
else if (data.liabilityShift === "POSSIBLE") → capturar
else → no capturar
```

21.4 Momento 4 — Captura

```
POST /api/orders/<ORDER_ID>/capture + body._cmid = cmid
→ backend inyecta:
  PayPal-Request-Id: <MISMO_UUID_DE_CREATE_ORDER>
  PayPal-Client-Metadata-Id: <CMID>
```

21.5 Regla mnemotécnica

Componente	Frecuencia
CMID + Fraudnet	Una vez por sesión de checkout.
STC + PayPal-Client-Metadata-Id	En cada intento de cobro (tarjeta nueva o guardada).
PayPal-Request-Id	Mismo UUID para Create Order y su Capture asociado; uno nuevo por transacción.

22. Caso de Uso Especializado — BOPIS

BOPIS: el comprador paga en línea pero recoge el pedido en una tienda física. El objeto shipping se reemplaza con los datos del punto de pickup, no del comprador.

Campo	Delivery (entrega a domicilio)	BOPIS (pickup en tienda)
shipping.name.full_name	<NOMBRE_DEL_COMPRADOR>	S2S <NOMBRE_DE_LA_TIENDA>
shipping.address	Dirección del comprador	Dirección física de la tienda

```
"shipping": {
  "name": { "full_name": "S2S <NOMBRE_DE_LA_TIENDA>" },
  "address": {
    "address_line_1": "<CALLE_Y_NUMERO_DE_LA_TIENDA>",
    "address_line_2": "<COLONIA_O_REFERENCIA>",
    "admin_area_2": "<CIUDAD_O_MUNICIPIO>",
    "admin_area_1": "<ESTADO_CODIGO>",
    "postal_code": "<CODIGO_POSTAL>",
    "country_code": "<PAIS_ISO_ALPHA2>"
  }
}
```

💡 El prefijo S2S (Ship To Store, mayúsculas, con espacio después) permite a PayPal Risk identificar el flujo como pickup en tienda.

23. Puntos de Integración (Mapa de Endpoints)

Operación	Backend del comercio (sugerido)	API de PayPal
Token OAuth para SDK	GET /api/token	POST /v1/oauth2/token
Crear orden	POST /api/orders	POST /v2/checkout/orders
Capturar orden	POST /api/orders/:id/capture	POST /v2/checkout/orders/{id}/capture
Consultar orden	GET /api/orders/:id	GET /v2/checkout/orders/{id}
Listar tokens guardados	GET /api/vault/payment-tokens?customer_id=...	GET /v3/vault/payment-tokens
Eliminar token	DELETE /api/vault/payment-tokens/:id	DELETE /v3/vault/payment-tokens/{id}
Opciones de financiamiento (token)	POST /api/credit/financing-options	POST /v1/credit/calculated-financing-options
Set Transaction Context	POST /api/stc/:cmid	PUT /v1/risk/transaction-contexts/{merchant_id}/{cmid}

24. Consideraciones de Seguridad

24.1 Custodia de credenciales (NFR-02)

Asset	Almacenamiento	Transmisión
PAYPAL_CLIENT_SECRET	Variable de entorno del backend, gestionada por un secrets manager. Nunca en código fuente.	Solo backend → /v1/oauth2/token codificado en Authorization: Basic.
PAYPAL_MERCHANT_ID	Variable de entorno del backend.	Solo backend → URL de STC.
access_token	Memoria del proceso del backend. Caché con TTL inferior al expires_in. Nunca persistido.	Solo backend → API REST de PayPal en Authorization: Bearer.
id_token	Generado dinámicamente, transmitido al frontend por respuesta a GET /api/token.	Backend → frontend → SDK (atributo data-sdk-client-token).

24.2 Aislamiento PCI / CSP / Logging

- Aislamiento PCI (NFR-01): Card Fields renderiza los inputs como iframes del dominio paypal.com. El PAN y el CVV nunca entran al DOM del comercio. El comercio puede certificarse bajo SAQ A.
- Transporte (NFR-03): TLS 1.2 mínimo en producción.
- CSP mínima: script-src 'self' https://www.paypal.com https://www.paypalobjects.com https://c.paypal.com; frame-src https://www.paypal.com; connect-src 'self' https://api-m.paypal.com https://api-m.sandbox.paypal.com
- Permitido en logs: order_id, capture_id, invoice_id, custom_id, customer.id, CMID, PayPal-Request-Id, liabilityShift, enrollment_status, authentication_status, códigos de error de PayPal.
- Prohibido en logs: PAN, CVV, fecha de expiración completa, nombre del titular en formato libre, access_token, CLIENT_SECRET, header Authorization completo.

25. Consideraciones Operativas

25.1 Métricas y observabilidad

Métrica	Granularidad	Alarma sugerida
Tasa de éxito de POST /v1/oauth2/token	Por minuto	< 99% en ventana de 5 min.
Latencia p95 de Create Order y Capture Order	Por minuto	Excede SLO interno.
Tasa de liabilityShift !== "POSSIBLE" (cuando 3DS se activó)	Por hora	Anomalía vs baseline histórico.
Tasa de respuesta ≠ 200 en STC	Por minuto	> 5% en ventana de 5 min.
Tasa de UNPROCESSABLE_ENTITY en Create Order	Por minuto	> 0.5% indica bug en construcción del breakdown.
Inventario de Fraudnet inyectado en el DOM	Por sesión	Más de 2 scripts inyectados → bug.

25.2 Tolerancia a fallos y promoción a Live

Servicio	Estrategia
POST /v1/oauth2/token	Reintentar una vez tras 401/5xx; si persiste, abortar y alertar.
POST /v2/checkout/orders	Reintentar con el mismo PayPal-Request-Id ante 5xx o errores de red transitorios.
PUT /v1/risk/transaction-contexts/... (STC)	No reintentar. Loguear el fallo y continuar el checkout.
https://c.paypal.com/da/r/fb.js (Fraudnet)	Si el script no carga, continuar el checkout (degradación elegante, riesgo aumenta).

Punto	Cambio para producción
PAYPAL_API_BASE	https://api-m.sandbox.paypal.com → https://api-m.paypal.com
PAYPAL_CLIENT_ID / PAYPAL_CLIENT_SECRET	Credenciales de la app Live, no Sandbox.
PAYPAL_MERCHANT_ID	Merchant ID de la cuenta Live.

Punto	Cambio para producción
application_context.return_url / cancel_url	URLs reales del dominio de producción del comercio.
additional_data de STC	Datos del usuario autenticado real, no valores de prueba.

Parte V — Validación y gobierno

26. Estrategia de Pruebas y Tarjetas de Sandbox

26.1 Tarjetas para Vault y tokenización (Sandbox)

Marca	Número	Notas
Visa (Bancomer)	4772129056533503	Tarjeta de crédito emitida por Bancomer (México).
Mastercard	5288775404117508	Tarjeta de crédito.
Amex	376680816376961	CVV de 4 dígitos.

26.2 Tarjetas para 3DS Risk Initiated

 Para activar 3DS en Sandbox se requieren dos condiciones simultáneas: (1) Card Holder Name = 3dsuser (literal, exactamente). (2) Número de tarjeta de las tablas de §26.2.1 o §26.2.2. Si falta cualquiera de las dos, el motor de riesgo de PayPal no lanza 3DS.

26.2.1 Flujos Frictionless (sin modal al usuario)

Caso	Marca	liabilityShift	enrollment	auth	¿Capturar?	Número
Frictionless exitoso	Mastercard	POSSIBLE	Y	Y	SÍ	5445492013842209
Frictionless exitoso	Visa	POSSIBLE	Y	Y	SÍ	4401331018783148
Frictionless fallido	Mastercard	N	Y	N	NO	5445492022569124
Frictionless fallido	Visa	N	Y	N	NO	4401331026683975
Attempts Stand-In	Mastercard	POSSIBLE	Y	A	SÍ	5445492038663051
Attempts Stand-In	Visa	POSSIBLE	Y	A	SÍ	4401331039804212

Caso	Marca	liabilityShift	enrollment	auth	¿Capturar?	Número
Auth no disponible (emisor)	Mastercard	N	Y	U	NO	5445492048677687
Auth no disponible (emisor)	Visa	N	Y	U	NO	4401331042569984
Auth rechazada por emisor	Mastercard	N	Y	R	NO	5445492055763685
Auth rechazada por emisor	Visa	N	Y	R	NO	4401331055071902
Auth no disponible en lookup	Mastercard	POSSIBLE	Y	A	SÍ	5445492061636883
Auth no disponible en lookup	Visa	POSSIBLE	Y	A	SÍ	4401331066329091

26.2.2 Flujos Step-Up (con modal al usuario)

Caso	Marca	liabilityShift	enrollment	auth	¿Capturar?	Número
Step-Up exitoso	Mastercard	POSSIBLE	Y	Y	SÍ	5445492100342725
Step-Up exitoso	Visa	POSSIBLE	Y	Y	SÍ	4401331109711123
Step-Up fallido	Mastercard	N	Y	N	NO	5445492119435767
Step-Up fallido	Visa	N	Y	N	NO	4401331117299038
Step-Up no disponible	Mastercard	N	Y	U	NO	5445492122987739
Step-Up no disponible	Visa	N	Y	U	NO	4401331128022452

26.3 Diccionario de valores

Valor	Significado	liabilityShift → acción
Y	Autenticación exitosa y verificada.	POSSIBLE → capturar.
N	Autenticación fallida.	N → no capturar.
A	Attempts — intento sin garantía completa.	POSSIBLE → capturar.
U	No disponible — banco/sistema no pudo completar.	N → no capturar.
R	Rechazada explícitamente por el emisor.	N → no capturar.

26.4 Casos de prueba mínimos (TC-01 a TC-14)

ID	Caso	Resultado esperado	Cumple RF
TC-01	Pago con tarjeta nueva, sin 3DS, sin Vault	Captura exitosa, token NO aparece en Vault.	RF-01, RF-02
TC-02	Pago con tarjeta nueva, 3DS Frictionless exitoso	liabilityShift = "POSSIBLE", captura exitosa.	RF-03
TC-03	Pago con tarjeta nueva, 3DS Step-Up exitoso	Modal aparece, captura exitosa al aprobar.	RF-03
TC-04	Pago con tarjeta nueva, 3DS Step-Up fallido	Modal aparece, NO captura, UI muestra error claro.	RF-03, RF-10
TC-05	Vault-with-Purchase	Captura exitosa, token aparece en GET /v3/vault/payment-tokens.	RF-04
TC-06	Pago con tarjeta guardada (token)	Captura exitosa o auto-captura, según status.	RF-05
TC-07	Eliminación de token	DELETE retorna 204, token desaparece del listado.	RF-06
TC-08	Pago a 3 MSI con tarjeta nueva	Selector muestra "3 meses sin intereses", captura exitosa.	RF-07
TC-09	Pago a 6 MCI con tarjeta nueva	Selector muestra cargo, captura exitosa con fee_reference_id.	RF-07
TC-10	Pago a 6 MSI con tarjeta guardada	calculated-financing-options devuelve la opción, captura exitosa.	RF-05, RF-07
TC-11	STC responde 400	Checkout no se interrumpe; orden se crea correctamente.	RF-08

ID	Caso	Resultado esperado	Cumple RF
TC-12	Inspección DOM al inicializar	Exactamente 2 scripts de Fraudnet (config JSON + fb.js).	RF-09
TC-13	Reintento dentro de la misma sesión tras fallo	Mismo CMID, misma carga de Fraudnet.	RF-12
TC-14	BOPIS (si aplica)	shipping.name.full_name con prefijo S2S.	RF-11

27. Convenciones de Nombres REST vs SDK

La inconsistencia entre `snake_case` (REST) y `camelCase` (SDK) es la causa más frecuente de bugs en Installments y Vault.

Concepto	API REST (<code>snake_case</code>)	SDK JavaScript (<code>camelCase</code>)
Duración del intervalo de financiamiento	<code>interval_duration</code>	<code>intervalDuration</code>
Referencia de comisión de la opción	<code>fee_reference_id</code>	<code>feeReferenceId</code>
País de financiamiento	<code>financing_country_code</code>	<code>financingCountryCode</code>
País de facturación	<code>billing.country_code</code> (objeto)	<code>billingCountryCode</code>
Código de moneda	<code>currency_code</code>	<code>currencyCode</code>
Resultado de 3DS en el callback	(no aplica en REST)	<code>liabilityShift</code>
Identificador de orden	<code>id</code> (en respuesta)	<code>orderId</code> (en callbacks)

 Regla mental: Si el dato sale o entra de un endpoint REST, es `snake_case`. Si lo pasas a un método del objeto `paypal.*` o lo recibes en un callback, es `camelCase`.

28. Asunciones, Dependencias y Restricciones

28.1 Asunciones

ID	Asunción
A-01	El comercio dispone de un backend bajo su control donde puede custodiar PAYPAL_CLIENT_SECRET, PAYPAL_MERCHANT_ID y emitir el access_token.
A-02	El comercio tiene un sistema de autenticación de usuarios que produce un customer.id estable y determinista por comprador.
A-03	El comercio tiene un servicio de carrito que produce un breakdown matemáticamente consistente.
A-04	El mercado destino soporta la moneda configurada y las modalidades MSI/MCI aplicables.
A-05	El comprador usa un navegador moderno con soporte de iframes, ES2017+ y CSP. La degradación para navegadores legacy se gestiona vía cardField.isEligible().
A-06	La cuenta de PayPal tiene activadas las habilitaciones comerciales listadas en §10.1.

28.2 Dependencias externas

ID	Dependencia	Tipo
D-01	API REST de PayPal (api-m.paypal.com, api-m.sandbox.paypal.com).	Crítica — bloqueante.
D-02	JavaScript SDK de PayPal (https://www.paypal.com/sdk/js).	Crítica — bloqueante.
D-03	Stylesheet base de Card Fields.	Recomendada.
D-04	Script de Fraudnet (https://c.paypal.com/da/r/fb.js).	Recomendada — degradación elegante si no carga.
D-05	Endpoint OAuth2 (/v1/oauth2/token).	Crítica — sin él no hay access_token ni id_token.
D-06	Endpoint STC (/v1/risk/transaction-contexts).	Recomendada — no bloqueante.

28.3 Restricciones

ID	Restricción
R-01	3DS Risk Initiated solo opera con cuentas que tengan CBP activo. Cuentas con Fraud Protection (FP) requieren la modalidad Merchant Initiated, fuera del alcance de este SDD.
R-02	El customer.id en Vault debe ser estable; cambiarlo invalida la asociación con tokens previamente guardados.
R-03	El CMID es único por sesión de checkout y no debe reutilizarse entre compradores ni sesiones distintas.
R-04	PayPal-Request-Id debe ser idempotente: el mismo UUID para Create Order y su Capture asociado; nuevo por transacción.
R-05	Los scripts de Fraudnet deben inyectarse en el orden correcto: primero la configuración JSON, luego fb.js.
R-06	El data-client-metadata-id del SDK debe coincidir exactamente con el "f" de Fraudnet, la URL de STC y el header de Create/Capture Order.

29. Riesgos y Mitigaciones

ID	Riesgo	Prob.	Impacto	Mitigación
RG-01	Exposición de PAYPAL_CLIENT_SECRET por commit accidental al repositorio.	Media	Crítico	Pre-commit hooks; secrets manager; revisión de seguridad obligatoria. Plan de rotación inmediata.
RG-02	Exposición de access_token al frontend por bug en GET /api/token.	Baja	Crítico	Tests automatizados que validen que la respuesta no contiene access_token; revisión de código.
RG-03	Inyección múltiple de Fraudnet por bug de carga.	Media	Medio	Bandera booleana a nivel de módulo (§13.3); test de inspección DOM en QA.
RG-04	Desincronización del CMID entre Fraudnet, SDK, STC y headers.	Media	Alto	Generar el CMID en un único punto y propagarlo por referencia; validación con regex (§12.3).
RG-05	Captura de pagos con liabilityShift desfavorable por bug en onApprove.	Baja	Crítico	Tests automatizados que cubran la matriz de §20.3; alarma operativa.
RG-06	Order duplicada por reintento sin idempotencia.	Media	Alto	PayPal-Request-Id consistente entre reintentos.
RG-07	Mezcla snake_case/camelCase en Installments → UNPROCESSABLE_ENTITY.	Alta	Medio	Tabla de convenciones (§27); revisión de código; tests específicos para MSI y MCI.
RG-08	Carga de credenciales del entorno equivocado (Sandbox en Live).	Baja	Crítico	Validación del prefijo del CLIENT_ID y del PAYPAL_API_BASE al arrancar el backend.
RG-09	Caída del SDK de PayPal o de Fraudnet por incidente externo.	Baja	Alto	Monitoreo activo de los dominios PayPal; mensaje de degradación al usuario.
RG-10	Bloqueo del checkout por error de STC (incumplimiento de NFR-05).	Baja	Crítico	STC implementado como no bloqueante con try/catch y respuesta 200 garantizada al frontend. Test TC-11.
RG-11	CSP demasiado restrictiva bloquea Fraudnet o SDK.	Media	Alto	CSP probada en staging; fallback de detección y mensaje claro al usuario.

ID	Riesgo	Prob.	Impacto	Mitigación
RG-12	Datos hardcoded de demo persisten en producción.	Media	Alto	Code review obligatorio; checklist de §30; búsqueda de strings literales sospechosos.

30. Criterios de Aceptación y Checklist Pre-producción

30.1 Seguridad

- ☐ PAYPAL_CLIENT_SECRET reside únicamente en variables de entorno del backend; nunca se incluye en código fuente versionado ni se envía al navegador.
- ☐ Archivos de configuración con credenciales (.env, equivalentes) excluidos del control de versiones.
- ☐ El backend solo expone clientId e idToken al frontend; nunca access_token.
- ☐ Logs no registran PAN, CVV, access_token ni headers Authorization.
- ☐ Content Security Policy permite <https://www.paypal.com>, <https://www.paypalobjects.com> y <https://c.paypal.com>.
- ☐ TLS 1.2+ habilitado en producción.

30.2 Configuración

- ☐ PAYPAL_API_BASE apunta a <https://api-m.paypal.com> en Live.
- ☐ PAYPAL_CLIENT_ID, PAYPAL_CLIENT_SECRET y PAYPAL_MERCHANT_ID corresponden al entorno Live.
- ☐ application_context.return_url y cancel_url son URLs reales y accesibles del dominio del comercio.
- ☐ El atributo data-client-metadata-id del SDK usa el CMID generado dinámicamente, no un literal.
- ☐ El parámetro components del SDK incluye únicamente card-fields.

30.3 Funcional

- ☐ customer.id proviene del usuario autenticado en la plataforma del comercio, no de un literal.
- ☐ breakdown e items reflejan el estado real del carrito y cumplen las reglas de §17.3.
- ☐ PayPal-Request-Id: el mismo UUID en Create Order y su Capture asociado; uno nuevo por transacción.
- ☐ Manejo de errores implementado en onError y en onInstallmentsError.
- ☐ La UI maneja explícitamente liabilityShift !== "POSSIBLE" cuando está presente.
- ☐ Los datos de payer, shipping y purchase_units provienen del estado del checkout, no de placeholders.

30.4 Riesgo

- ☐ CMID generado una sola vez por sesión de checkout.
- ☐ Fraudnet inyectado una sola vez (campo "f" con el CMID dinámico).
- ☐ Campo "s" de Fraudnet con formato <NOMBRE_CORTO>_<MERCHANT_ID>_ACDC.

- ☐ STC se llama antes de cada Create Order (tarjeta nueva y guardada).
- ☐ additional_data de STC proviene de la sesión autenticada del comprador.
- ☐ STC no bloquea el checkout: errores se loguean y la transacción continúa.
- ☐ Header PayPal-Client-Metadata-Id presente en Create Order y Capture Order, con el mismo CMID.

30.5 Pruebas

- ☐ Matriz de casos de prueba TC-01 a TC-14 ejecutada y verde en Sandbox.
- ☐ Inspección del DOM confirma exactamente dos scripts de Fraudnet (config + fb.js).
- ☐ STC con PUT /v1/risk/transaction-contexts/... responde 200 antes de cada Create Order.

30.6 Operación

- ☐ Métricas de §25.1 instrumentadas y conectadas al sistema de observabilidad.
- ☐ Alarmas configuradas para los umbrales sugeridos.
- ☐ Runbook de incidentes documentado: falla de OAuth, falla de Create Order, anomalía de liabilityShift.
- ☐ Plan de rotación de CLIENT_SECRET documentado y probado en Sandbox.

Apéndice A — Diagnóstico de Errores Frecuentes

Síntoma	Causa más probable	Cómo diagnosticar
401 Unauthorized en /v1/oauth2/token	CLIENT_ID / CLIENT_SECRET mal copiados o entorno cruzado.	Verificar que PAYPAL_API_BASE y las credenciales correspondan al mismo entorno.
422 UNPROCESSABLE_ENTITY en Create Order	breakdown que no cuadra con amount.value y/o suma de items.	Aplicar reglas de §17.3; PayPal devuelve el campo inconsistente en la respuesta.
422 UNPROCESSABLE_ENTITY con installments MCI	fee_reference_id faltante.	En MCI fee_reference_id es obligatorio en submit (camelCase) o en attributes.installments (snake_case).
MSI y MCI se mezclan en la UI	Solo se mira term, no total_consumer_fee.	Usar parseFloat(opt.total_consumer_fee.value) === 0 para clasificar.
liabilityShift === "N" y se intenta capturar	onApprove no valida el resultado de 3DS.	Implementar la tabla de decisión de §20.3.
Token guardado no aparece al listar	customer.id distinto entre Vault-with-Purchase y consulta.	El customer.id debe ser el mismo identificador estable del comprador en ambos puntos.
STC responde 400	Tipo de campo incorrecto en additional_data.	highrisk_txn_flag debe ser number (0 o 1), no string. sender_create_date debe usar uno de los formatos permitidos.
STC responde 401	MERCHANT_ID incorrecto, access_token expirado o sin permiso.	Verificar PAYPAL_MERCHANT_ID. Refrescar token. Validar habilitación con el IE.
Fraudnet se inyecta múltiples veces	Falta la bandera fraudnetLoaded.	Implementar el patrón booleano de §13.3 a nivel de módulo.
data-client-metadata-id aparece vacío	El SDK se cargó antes de generar el CMID.	Garantizar el orden estricto: §12 → §13 → §11 → §15.
Funciona en Postman pero falla en el navegador	El frontend está intentando llamar a api-m.paypal.com directamente.	El frontend siempre va contra /api/* del backend; nunca directo a la API REST de PayPal.
Card Fields no renderiza	cardField.isEligible() devuelve false o el SDK no se cargó.	Validar isEligible() antes de .render(). Revisar la consola y la pestaña Network.

Síntoma	Causa más probable	Cómo diagnosticar
3DS nunca se activa en Sandbox	Falta el nombre 3dsuser o tarjeta no listada.	Cumplir simultáneamente las dos condiciones de §26.2.

Apéndice B — Limitaciones y Trabajo Futuro

B.1 Limitaciones conocidas

- 3DS Merchant Initiated no está cubierto. Comercios con Fraud Protection (FP) requieren una solución distinta basada en `payment_source.card.attributes.verification`.
- Las tarjetas de prueba y comportamientos descritos en §26 son los publicados por PayPal Sandbox al momento de redactar este SDD. PayPal puede actualizar el set de prueba; consultar la documentación oficial vigente ante discrepancias.
- Los industry packs de STC (Travel, OTAs, Gaming, Financial Services, etc.) requieren campos adicionales no incluidos en el set genérico Retail de §14.2. Solicitar el pack correspondiente al Integration Engineer.
- La divergencia REST vs SDK (`snake_case` vs `camelCase`) puede cambiar entre versiones del SDK. La versión cargada del SDK es la fuente de verdad para los nombres de los callbacks.
- Esta solución describe la integración server-to-server del comercio. Webhooks, reembolsos, settlement y disputas requieren un SDD complementario.

B.2 Trabajo futuro sugerido

Iniciativa	Descripción
Webhooks	SDD complementario para suscripción y procesamiento de eventos asíncronos (PAYMENT.CAPTURE.COMPLETED, PAYMENT.CAPTURE.DENIED, CUSTOMER.DISPUTE.CREATED, etc.).
Reembolsos	Diseño del flujo de back-office para reembolsos parciales y totales (POST /v2/payments/captures/{id}/refund).
Conciliación financiera	Integración con reportes de PayPal y settlement para reconciliación contable.
APMs adicionales	Extensión de la solución para incluir Pay Later, Venmo o billeteras locales.
Industry pack	Adopción del set extendido de <code>additional_data</code> de STC correspondiente a la vertical del comercio.

Solution Design Document para la integración de PayPal ACDC con Card Fields, 3DS Risk Initiated, Installments (MSI/MCI), Vault, Fraudnet y Set Transaction Context (STC).